最小可用“三审制”稿件工作台

一、 业务背景

在媒体内容生产系统中，“三审制”（一审、二审、三审）是保障内容质量与安全的核心工作流。现在实现一个可运行的微型全栈应用，模拟一篇稿件从创建、编辑、提交审核，经过三审处理，直至最终审核完成的完整生命周期。

目标：不追求功能的堆砌，但要求体现出完整的业务闭环、清晰的架构设计以及良好的工程代码。请优先完成**最小可用版本（MVP）**。

二、 核心流程与状态机

1. 状态流转规则

系统中至少需要体现以下状态（你可以自行优化命名，但需覆盖核心语义）：

- `草稿`（Draft）
- `待一审`（Pending 1st Review）
- `待二审`（Pending 2nd Review）
- `待三审`（Pending 3rd Review）
- `已退回`（Rejected）
- `已完成`（Completed/Published）

2. 基本流转路径

- **正向流程**：草稿 -> 待一审 -> 待二审 -> 待三审 -> 已完成。
- **退回流程**：在待一审、待二审、待三审的任一环节，审核人均可执行“退回”操作，状态变更为“已退回”。
- **重新提交**：已退回的稿件支持重新编辑并再次提交审核（说明：退回后重新提交，回到“退回前的上一级环节”）。

三、 核心功能要求

1. 稿件管理模块

- **创建与编辑**：支持输入标题和正文。

- **保存与提交**：支持保存为草稿，或直接提交进入审核流。
- **稿件列表**：展示稿件标题、当前状态、更新时间等基本信息。
- **稿件详情**：展示稿件全部内容。

2. 审核处理模块

- **审核流转**：能够清晰看到稿件当前所处状态。
- **审核操作**：支持“审核通过”与“审核退回”。**执行退回操作时，必须填写退回意见。**
- **审核记录**：稿件详情页需展示基本的历史审核记录（什么时间、做了什么操作、意见是什么），保证流转有迹可循。

3. 界面组织（前端要求）

- 需包含基本的页面或模块切换（如：列表页、编辑页、详情/审核页）。
- 根据稿件当前状态，提供相应的操作按钮。例如：草稿状态仅可编辑/提交；待审核状态仅可展现通过/退回操作；已完成状态仅可查看。

4. 接口与数据支撑（后端要求）

- 提供支撑上述前端能力的后端服务接口。
- **数据存储**：可以使用任一你熟悉的关系型数据库。

四、 进阶挑战（加分项）

说明：以下功能并非强制要求。请在确保主流程跑通、代码结构清晰的前提下，根据自身余力选做。

1. **账户与权限（RBAC）**：实现简单的用户体系，通过角色（作者、一审员、二审员等）控制操作权限。
2. **富文本与素材**：集成富文本编辑器，实现稿件中图片、视频等素材资源的存储。
3. **检索与性能**：支持稿件列表的搜索、分页，或简单全文检索。
4. **发布解耦**：预留第三方发布接口 Hook，支持审核结束后模拟对接外部平台。
5. **消息通知**：设计内部消息通知路由（例如：审核通过后自动提醒下一级审核人）。
6. **AI 辅助能力**：引入 AI 辅助功能，例如自动提取稿件关键词、生成摘要，或辅助审核人员进行错别字/语法校验。

五、 技术规范与交付

1. 技术栈

不限。请使用你最熟悉的语言和框架（如 Vue/React + SpringBoot/Node.js/Go 等）。允许使用 AI 辅助开发。

2. 交付内容

请将项目打包或提供代码仓库链接，必须包含：

- **前后端源代码**：代码需结构清晰、命名规范，包含必要的注释。
- **启动说明文档（README）**：提供明确的依赖安装与启动方式。
- **系统设计说明**：
  - 简述前后端交互的 API 接口设计。
  - 简述数据模型设计方案。
  - 说明你的状态流转设计思路。